

摘要：

Architect Labs的AI系统两周内自主设计验证了AI加速器Redwood，仅需两位人类架构师提供规范。其性能达Jetson Orin Nano的1.75倍，功耗降低1.9倍。该系统实现了软硬件协同设计，并展示了递归式自改进。

日前，由 Ebrahim Hussain 和 Aaditya Subedi 领导的定制芯片人工智能研究实验室Architect Labs宣布，其人工智能系统根据人工编写的规范生成并完全验证了端到端的、可用于生产的人工智能加速器 Redwood。

仅由两位人类架构师提供技术规范，人工智能系统就在不到两周的时间内完成了芯片的设计和全面验证。它还参与了固件和定制内核的设计，并将现代人工智能模型映射到硬件上。最终的加速器目前运行在FPGA平台上，对包括Llama和Qwen在内的数十亿参数模型进行推理。

据相关人士所说，Redwood 代表着半导体行业的一个重要里程碑：一个人工智能系统自主设计出了一款可运行人工智能模型的量产型人工智能芯片。这种方法在人工智能模型和为其优化的硬件之间建立了一个反馈回路，有望将芯片设计速度提升到超越传统开发周期的水平。

性能测试结果也表明，Redwood 不仅仅是概念验证。基于三星的 8nm 工艺，Redwood 的吞吐量是 NVIDIA Jetson Orin Nano 的 1.75 倍，功耗却降低了 1.9 倍，这意味着在相同的 AI 模型下，其每瓦性能提升了 3.4 倍。

芯片设计可能需要数年时间和数亿美元，而专业人才的日益匮乏使得先进半导体研发集中在少数几家大型公司手中。因此，人工智能工作负载往往需要适配远在最新型号问世之前设计的硬件。

Redwood 采取了不同的方法，从一开始就将硬件和软件进行协同设计。内核、固件和 RTL 代码共同开发和优化，使芯片能够根据 AI 工作负载进行定制，而不是强迫软件去适应现有的硬件。

这形成了一个持续的反馈循环，改进的 AI 模型可以为更好的硬件设计提供信息，而更高效的硬件可以实现更好的 AI 性能，从而有可能加速开发周期的两端。

“四十年前我刚入行时，芯片设计只需要一两个工程师。从那时起，设计的复杂性、耗时和风险都呈指数级增长。即使EDA工具和技术不断进步，一个芯片项目仍然需要耗费数年时间和庞大的工程团队，”英特尔前工程高级副总裁Sunil Shenoy说道。“Redwood真正实现了范式转变，树立了技术前沿的标杆。Architect Labs正在以我以前难以想象的速度，将曾经只有少数巨头才能掌握的能力普及化。他们的方法有望将硬件带回未来。”

走进Redwood：前沿人工智能加速器

Redwood 是一个端到端的 AI 推理平台，专为需要在严格的功耗限制下实现实时性能的物理 AI 应用（例如机器人、无人机和边缘设备）而设计。

其核心是一个可扩展的矩阵和向量计算引擎网格，这些引擎通过专用的片上网络连接起来。完整的AI推理流程，包括注意力机制、键值缓存和即时量化，都直接在芯片上运行，无需依赖主机处理器。

计算引擎、网络、固件和定制内核作为一个统一的系统进行设计和优化，使硬件能够紧密匹配现代人工智能工作负载。Redwood 还可以扩展到更大的数据中心 SoC，或作为独立的芯片组运行。

关于自主Redwood设计的关键统计数据：

- 自主设计和验证： 100% 的 RTL、UVM 验证环境、形式验证、固件、驱动程序和自定义计算内核均由 Architect Labs 的 AI 系统根据人工编写的规范在不到两周的时间内端到端生成，而该项目由两名人类架构师参与。
- 达到签核级验证标准，硬件零缺陷：从单个IP到整个SoC，每个模块的代码和功能覆盖率均超过95%，验证过程使用了商用EDA工具、Architect Labs专有的形式化验证引擎以及硬件在环验证。从仿真到FPGA平台的首批RTL代码均未发现任何缺陷。
- 在真实硬件上运行真实的AI模型： Redwood Nano部署在AMD Versal FPGA上，运行频率为250MHz，可对包括Qwen在内的开放权重模型执行实时单批次推理。Architect Labs在今年的设计自动化大会（DAC）上进行了现场硬件演示，是展会上为数不多能够展示AI设计的加速器并实时运行模型推理的公司之一。
- 超越当今领先的AI芯片：在与NVIDIA Jetson Orin Nano采用相同工艺的三星8纳米工艺平台上，Redwood的吞吐量提升了1.75倍，功耗降低了1.9倍，每瓦性能提升了3.4倍（以运行相同模型的Jetson基准测试为基准）。这些预测结果基于FPGA的直接测量数据，而非仅基于仿真。
- 架构迭代以天为单位，而不是以季度为单位：对高级规范的任何更改都会导致硬件在 48 小时内完全重新生成、重新验证和重新部署，但受 EDA 工具运行时间限制的 SoC 级运行除外。
- 递归式自我改进：部署在 Redwood 上的 AI 模型以 API 端点的形式公开，发现了加速器本身的计时和内核优化，推理成本几乎为零，从而闭合了 AI 和驱动它的硅芯片之间的闭环。

“每个计算时代的进步都离不开底层硬件的支持，但也受限于谁有能力制造相应的芯片，” Kindred Ventures 的创始人兼管理合伙人 Steve Jang 表示。“Redwood 芯片的问世初步证明，这道门槛可以打破：两个人——从一份书面规范和目标 AI 模型入手——利用 Architect Labs 的系统，在短短几周内就完成了极具竞争力的 AI 加速器的设计、验证和部署。无论你是前沿研究实验室、机器人制造商还是云运营商，为你的产品或平台量身定制芯片的概念如今正逐渐成为现实。”

一种从根本上革新的硅芯片软件设计方法

Architect Labs 的 AI 系统能够并行优化整个计算堆栈，从模型和内核到固件和 RTL 代码，而非采用传统芯片设计中常见的顺序、孤立的工作流程。这使得软件和芯片能够在流片过程中进行协同优化，设计迭代时间可能从数月缩短至数周。

这种方法可以根据效率在软件和硬件之间切换功能，例如，用专用硬件逻辑替换数千个软件周期，或者将调度任务移至编译器。Redwood 证明，这些权衡取舍现在可以在几天内通过硬件完成设计、验证和测试，从而为将该方法扩展到更复杂的芯片奠定了基础。

“三十年前，像台积电这样的晶圆代工厂让任何拥有设计方案的人都能获得世界一流的制造能力，由此催生了以英伟达、博通和苹果等公司为代表的无晶圆厂半导体产业，”Architect Labs联合创始人兼首席执行官Ebrahim Hussain表示。“同样，我们正在引领无设计半导体产业的发展，像Redwood这样的芯片可以与运行的工作负载共同设计和演进。我们设想，拥有密集型工作负载或专用AI模型的软件公司可以获得共同设计的定制芯片，而无需组建庞大的设计团队，无需在架构上投入十年时间，也无需退而求其次使用现成的通用解决方案，从而节省性能、功耗和成本。每一项重要的工作负载都值得拥有专属的定制芯片。我们正在构建这样一个未来。”

Architect Labs表示，公司已将同样的方法应用于财富 500 强合作伙伴，以软件开发的速度共同设计定制芯片，并将原本需要数月才能运行的程序压缩到数周内即可完成。Redwood 是这项技术首次公开展示其成果。

附完整论文翻译：

Redwood: A Frontier AI Accelerator Designed, Verified, and Deployed from Scratch in 2 Weeks by AI

Architect Labs Team^{*}
Palo Alto, California

Abstract— Modern AI workloads and the hardware that runs them evolve on different timescales: architectural definition precedes volume silicon by years, while target workloads shift in months. Design decisions are therefore committed under deep uncertainty and paid for twice, once in the generality added as a hedge, and again when new workloads map poorly onto frozen silicon. As Moore’s Law stagnates, specialization is the main remaining source of performance-per-watt and demands a design cycle that runs at the cadence of the workloads. We present an end-to-end AI system that collapses the software-to-silicon stack into a single optimization loop, where hardware and software are co-designed and verified under one objective. Its first demonstration is Redwood, a frontier AI accelerator built for single-batch, low-power, ultra-low-latency inference for physical AI. From a high-level specification by two human architects, the system autonomously generated the performance model, RTL design, UVM environments, formal proofs, firmware, and kernels in under two weeks with no human intervention below the specification. Every block reached 95% coverage via commercial EDA tools, our proprietary formal engine, and hardware-in-the-loop validation. Specification changes were reverified and redeployed to hardware in under 48 hours. Redwood Nano, its ultra-low-power FPGA variant, runs multi-billion-parameter models like Llama and Qwen. Projected onto Samsung 8 nm, the Jetson Orin Nano’s process class, Redwood delivers 1.75x the throughput at 1.9x lower power, a 3.4x performance-per-watt gain against a measured Jetson baseline on the same models. Qwen running on Redwood also helped design next-generation Redwood, an early step toward recursive self-improvement. To our knowledge, this is the first production-worthy AI accelerator designed end-to-end by an AI system and running a modern AI model.

To address this, we introduce Redwood, a frontier AI accelerator designed, verified, programmed, and deployed end-to-end by such a system. Two human architects captured the workload and architectural constraints in a high-level specification. From that specification, the system autonomously generated the performance model, RTL, UVM environments, formal proofs, firmware, drivers, and custom compute kernels. In under two weeks, it produced the complete design from scratch, closed every block at 95% code and functional coverage, and deployed the Redwood Nano configuration on an AMD Versal FPGA. A third week brought Qwen3-0.6B inference online [6]. Each architectural change during this period was regenerated, reverified, and redeployed to hardware in under 48 hours. Evaluated on a Samsung 8 nm-class process comparable to that used by NVIDIA Jetson Orin Nano, Redwood Nano is projected to deliver 1.75x the decode throughput at 1.9x lower power, a 3.4x improvement in performance per watt, against the measured Jetson baseline running the same model.

The remainder of the paper is organized as follows. Section II describes the Redwood architecture. Section III presents the programming model. Section IV evaluates Redwood Nano on FPGA and projects its performance, area, and power. Section V details the Architect Labs AI system that produced the design, including automated verification, microarchitectural exploration, and firmware and kernel generation, and reports an early demonstration of recursive self-improvement of AI model and the hardware that fuels it.

现代人工智能工作负载及其运行所需的硬件以不同的时间尺度演进：架构定义比量产芯片早数年，而目标工作负载的变化却以月为单位。因此，设计决策是在高度不确定性下做出的，并且要付出双重代价：一次是为规避风险而增加的通用性，另一次是当新的工作负载难以映射到已冻结的芯片上时。

随着摩尔定律的停滞，专业化成为每瓦性能的主要来源，并要求设计周期与工作负载的节奏保持一致。我们提出了一种端到端的人工智能系统，它将软件到芯片的整个堆栈简化为一个单一的优化循环，其中硬件和软件在同一目标下进行协同设计和验证。该系统的首个演示是 Redwood，这是一款专为物理人工智能的单批处理、低功耗、超低延迟推理而构建的前沿人工智能加速器。该系统基于两位人类架构师提供的高级规范，在不到两周的时间内自主生成了性能模型、RTL 设计、UVM 环境、形式化证明、固件和内核，且规范以下部分无需人工干预。

通过商用EDA工具、我们自主研发的形式化引擎以及硬件在环验证，每个模块的覆盖率均达到 95%。规范变更在48小时内完成重新验证并部署到硬件。Redwood Nano是其超低功耗FPGA版本，可运行Llama和Qwen等数十亿参数模型。在三星8nm工艺（Jetson Orin Nano的工艺等级）下，Redwood的吞吐量提升了1.75倍，功耗降低了1.9倍，与在相同模型上测得的Jetson基准相比，每瓦性能提升了3.4倍。

在Redwood上运行的Qwen也助力了下一代Redwood的设计，这是迈向递归式自我改进的早期一步。据我们所知，这是首个由AI系统端到端设计并运行现代AI模型的、可用于生产环境的AI加



速器。

一、引言

EDA 行业目前报告称，人工智能 (AI) 在 RTL 生成、验证、调试和探索等方面带来了数量级的提升，包括将数月的工作量缩短至数天，并在设计和验证工作流程中实现高达 10 倍的生产力提升。然而，仅有 14% 的 IC/ASIC 项目实现了首芯片流片成功，这是近二十年来的最低水平，而 75% 的项目进度落后，因为芯片项目面临着日益复杂的设计挑战，这些挑战源于异构集成、先进节点物理效应以及日益严格的功耗、性能和面积 (PPA) 限制。这种差异凸显了任务级生产力声明与整个项目结果之间的严重不匹配：AI 加速了单个活动，但尚未在日益复杂的 SoC 上展现出明显的端到端项目改进。

与此同时，公开展示的端到端 AI 生成设计仍然局限于简单的示例，例如玩具 RISC-V 内核或强化的数值数据通路。几乎没有一项技术在物理硬件上得到验证，而物理硬件最终是硬件设计的最终约束。我们认为，人工智能在硬件设计中的应用机会并非在于现有流程中的任务加速，而在于对整个流程本身进行重新构想。当架构、RTL、验证、固件和内核都基于单一规范生成，并针对同一目标进行优化时，导致程序延迟的顺序交接环节就会消失，软硬件协同设计将成为系统本身的属性，而非团队之间的协调过程。

为了解决这个问题，我们推出了 Redwood，这是一款前沿的 AI 加速器，由该系统完成端到端的设计、验证、编程和部署。两位架构师在一份高级规范中记录了工作负载和架构约束。基于该规范，系统自主生成了性能模型、RTL 代码、UVM 环境、形式化证明、固件、驱动程序和自定义计算内核。在不到两周的时间内，系统从零开始完成了完整的设计，每个模块的代码和功能覆盖率均达到 95%，并在 AMD Versal FPGA 上部署了 Redwood Nano 配置。第三周，Qwen3-0.6B 推理上线。在此期间，每次架构变更都在 48 小时内重新生成、重新验证并重新部署到硬件上。Redwood Nano 采用与 NVIDIA Jetson Orin Nano 类似的 8 纳米级三星工艺进行评估，预计其解码吞吐量将提升 1.75 倍，功耗降低 1.9 倍，每瓦性能提升 3.4 倍（与运行相同模型的 Jetson 基准相比）。

二、架构

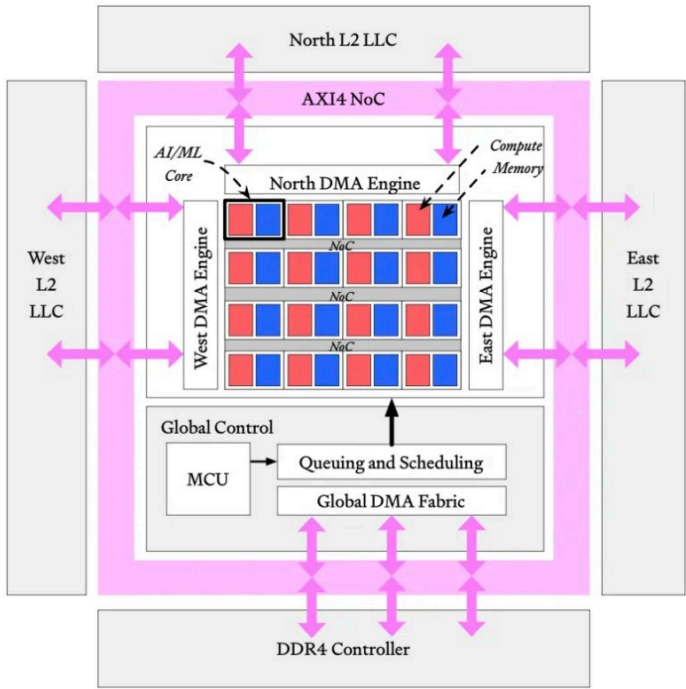


Fig. 1: Redwood SoC architecture.¹

Redwood 是一款基于 tile 的空间数据流加速器，它使用标准的 AXI4 内存映射接口：AXI-Lite 用于控制和配置，而支持完整突发传输的宽 AXI4 用于批量数据传输（图 1）。专用 DMA 引擎负责所有与外部 DRAM 之间的数据传输。全局 DMA (GDMA) 架构在外部存储器和片上西、北、东三个末级 SRAM 存储体 (LLC) 之间执行批量内存到内存的传输，而边缘 DMA 引擎则负责在计算架构上进行数据暂存。全局控制区域负责对加速器进行排序，并包含全局控制核心 (MCU)、全局任务管理器以及一个 48 位全局定时器 (HAC)，该定时器会广播到每个 tile 以进行时间隔离调度。该区域负责启动、协调和清理诸如 FlashAttention 和 GEMM/GEMV 等内核。由于内存接口仅限于模块化 DMA 引擎，Redwood 可以集成到更大的 SoC 中，也可以封装成独立的芯片。DMA 后端可以从 AXI4 重定向到 ACE 和 CHI 等协议，而不会影响计算架构。

计算架构是一个 $N \times M$ 的网格，由相同的 tile 组成，周围环绕着边缘 DMA 引擎。每个 tile 都包含一个基于 RISC-V 的 tile 控制核心 (CRV) 和专为 Transformer 推理而设计的计算引擎。矩阵引擎 (CMXM) 提供脉动 GEMM 和矩阵向量 (GEMV) 数据通路，并将数据流直接传输到向量引擎 (CVXM)，后者提供 SIMD、转置和浮点激活单元。宽大的分块暂存存储器最大限度地减少了 Redwood 内部的数据移动。计算引擎与内核软件协同设计，因此它们可以直接映射到主要的 Transformer 算子——注意力机制、GEMM、归一化和激活——并将 FlashAttention 和 GEMM 等内核作为硬件调度任务执行，而不是作为通用指令流执行。高带宽、内部设计的、基于信用机制的片上网络 (NoC) 可承载 tile 到 tile、DMA 到 tile 和 tile 到 DMA 的流量。它提供低开销的广播和组播、基于表的流重定向以及逐链路流量控制。

A. Tile 架构

Redwood 架构中的每个 tile 都分为前端 (FE) 和后端 (BE)，如图 2 所示。FE 负责控制和编程，而 BE 负责数据传输和计算。将稀疏控制与高带宽数据处理分离，使得 FE 能够在较低的时钟频率下运行，并且在某些情况下，FE 可以在内核执行期间关闭，从而实现显著的节能效果。内核软件运行在 tile 控制核心 (CRV) 上，而核心任务管理器 (CTM) 则连接 CRV 和 BE 功能单元，并协调跨多个可配置单元的任务。

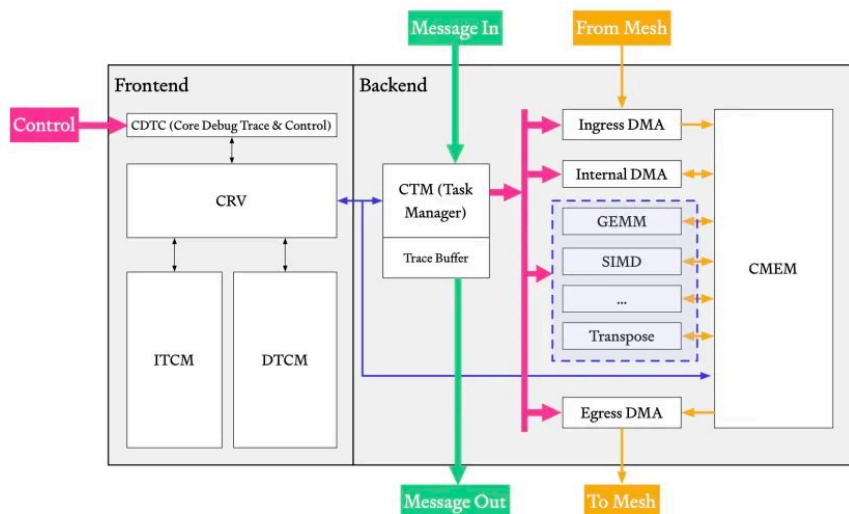


Fig. 2: Redwood tile architecture.

Tile BE 内部的硬件单元是为现代 Transformer 工作负载协同设计的，包括设备端预填充和解码。计算单元包括用于矩阵运算的 GEMV 和 GEMM 引擎（由阵列化的基于整数的乘加 (MAC) 单元构建），以及用于逐元素运算的多通道 SIMD 引擎（由阵列化的浮点单元 (FPU) 构建），能够处理归约、基于查找表 (LUT) 的运算等（图 3）。一项优化采用了 FlashAttention-4 中的模拟 softmax 算法，该算法重用了现有的 SIMD 资源来执行原本会占用大量面积的操作。这些功能单元和 CRV 通过高带宽总线共享对本地 512 KB 核心内存 (CMEM) 的访问。本地入口和出口 DMA 引擎负责将数据移入和移出 CMEM。

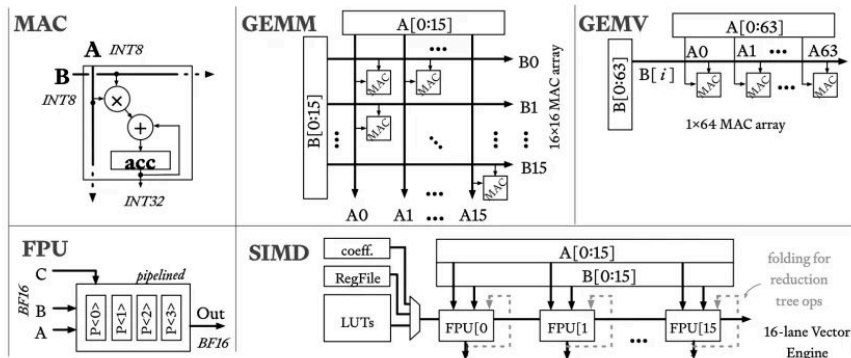


Fig. 3: Standard Redwood tile BE units.

B. 控制机制

要使用 Redwood tile 进行计算，首先需要将内核加载到前端指令紧耦合存储器 (ITCM：instruction tightly coupled memory) 中。然后，核心调试、跟踪和控制 (CDTC：Core Debug, Trace, and Control) 单元启动裸机 tile 控制核心 (CRV：control core)。CRV 等待 CDTC 向数据紧耦合存储器 (DTCM：data tightly coupled memory) 传递函数调用，然后执行选定的内核函数。内核函数通常会展开为多个 MMIO 写入操作，这些操作会将 CTM 任务排队，以便分发给相应的功能单元。只要内核函数的实现位于 ITCM 中，就可以将多个内核调用排队。整个前端-后端协调过程如图 4 所示。

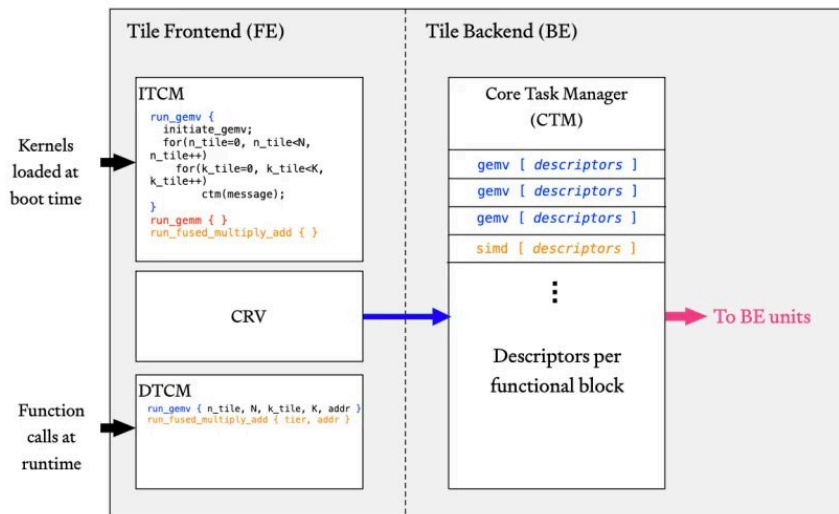


Fig. 4: Redwood FE-BE orchestration.

前端 (FE) 和后端 (BE) 之间的这种解耦带来了以下优势：

- CRV 保持简洁，实现了最小的 RISC-V 规范，面积和功耗开销都很低。
- 当添加、更改或移除 BE 功能单元时，CRV 解码器保持不变。
- CRV 可以将任务列表交给 CTM，然后保持空闲状态直到被中断。

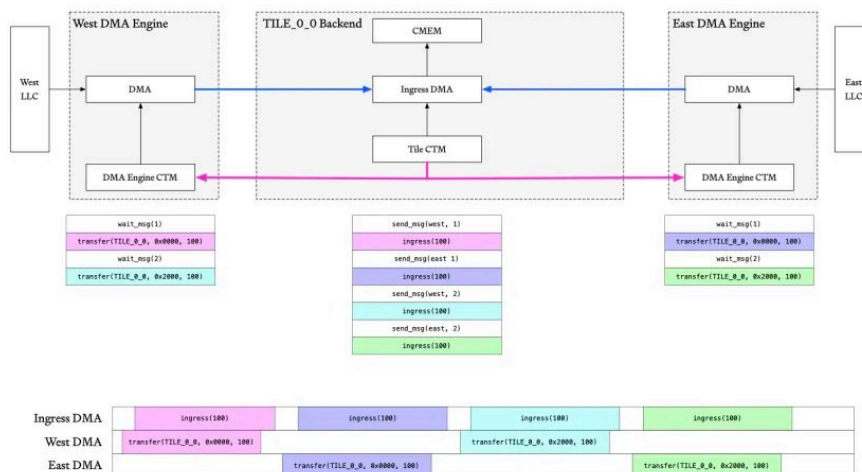
CTM 原生支持：

- 使用任务 ID 跟踪乱序完成情况，实现任意任务排序和隔离。

- 循环遍历队列任务的任意部分，以减少重复的 CRV 写入。

The diagram illustrates the Redwood Fabric Architecture. At the top, the Redwood Fabric consists of an Edge DMA, Edge Task Manager, and North L2 SRAM. Below this, a central AI/ML Core is shown, which contains a Tile CRV, Tile CTM, and Core DMAs. The core is connected to West and East L2 SRAMs, Edge DMAs, and Edge Task Managers. A Global Control block, containing a Global MCU, Global Task Manager, and Global DMA, is connected to the core via a green arrow and to the Redwood Fabric via a blue arrow. The diagram also shows a dashed line indicating N Cores.

消息机制允许 CTM 在不涉及 CRV 或 MCU 的情况下对控制流进行排序。在图 6 的典型示例中，位于 (0,0) 的 tile 向东西 DMA 引擎发送交错消息，它们的 CTM 会等待“允许”消息才能释放下一个任务。通信也可以反向进行，DMA CTM 向 tile CTM 发送信号，使其向下游发送数据。消息机制可以配置为“即发即弃”或“基于确认”。编译器使用消息机制来协调预取、双缓冲和乱序计算。通过消息进行显式流量控制减少了对网格中复杂仲裁的需求，并将调度移至软件栈中。



三、编程模型

Redwood 采用灵活的编程模型，可在同一架构上运行不同的模型和工作负载。全局控制核心 (MCU) 的程序称为调度程序 (DP)，而各个单元的程序称为内核。多个 DP 组成一个“DP 集”，多个内核组成一个“内核集”，从而分摊初始化开销。

主机处理器使用 Redwood 执行操作的流程如下（图 7）：

- 1.（前提条件）将 DP 集从外部存储器加载到 MCU 的 ITCM 中。
- 2.（前提条件）将内核集从外部存储器加载到所有 tile 的 ITCM 中。
3. 主机处理器将调度 ID 和操作数写入 MCU 的 DTCM，并向 MCU 发送信号。
4. MCU 执行由调度 ID 选择的调度程序。调度程序可以：
 - 对内部结构网格中的静态路由表进行编程。
 - 配置全局任务管理器以执行预取、内存到内存复制和分散/聚集操作。
 - 配置 DMA 引擎任务管理器以将数据移入和移出 tile 阵列。
 - 通过将内核 ID 和操作数写入 tile 的 DTCM 并向 tile 控制核心发送信号来启动 tile 内核。

DP 可以通过轮询状态或依赖 tile 中断来判断 tile 阵列何时完成。一个正在运行的 DP 在其生命周期内可能会启动一个或多个内核。例如，FlashAttention 会反复启动 tile 来处理不同的 KV 块和磁头。

5. 当 DP 完成时，MCU 会中断主机处理器，以指示执行已完成，并且预留的输入输出缓冲区可供主机使用。

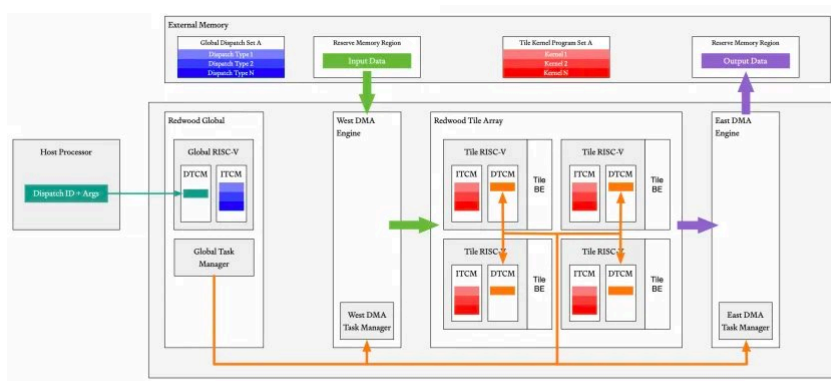


Fig. 7: Redwood dispatch and kernel execution flow.

如果整个模型所需的 DP 或内核集不适合 ITCM，则主机处理器会在运行时将它们加载到分区分中，并将它们分组以最大限度地减少交换。

四、评估

Redwood Nano 是 Redwood 的 FPGA 配置，专为超低功耗、低延迟边缘应用场景而设计和优化。它由一个 2×2 的 tile 阵列组成，每个 DMA 引擎都连接到一组 128 位 AXI4 接口。西侧和北侧接口优先考虑入口读取带宽，而东侧接口优先考虑出口写入带宽。Redwood Nano 在 250 MHz 的 AMD Versal VPK180 FPGA 上进行综合和部署（图 8）。为了评估性能，我们在 Redwood Nano 上运行 Qwen3-0.6B，并将其与 NVIDIA Jetson Orin Nano 进行比较。图 8 显示了 Redwood Nano 在 VPK180 FPGA 上的布局和实例化层次结构。

我们测量了峰值吞吐量和平均吞吐量下的 LLM 解码性能，单位为每秒输出的 token 数。对于 Redwood Nano，测量过程包括从主机向 FPGA 发送提示符、在 FPGA 上运行 Qwen 以及将每个输出 token 返回给主机。NVIDIA Jetson Orin Nano 运行的是相同型号的处理器的 GPU 时钟频率为 1020 MHz，性能测试使用 NVIDIA 的 Jetson WebUI 进行。表 I 列出了基准测试对比结果。

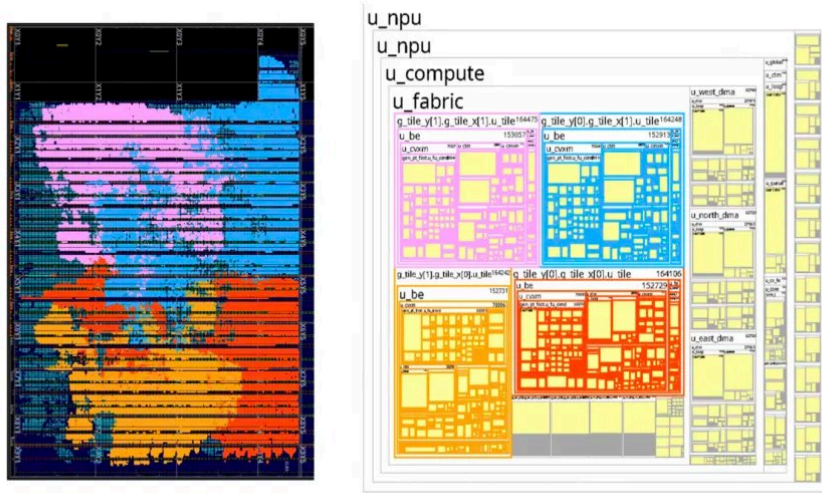


Fig. 8: Redwood Nano FPGA placement and instantiation hierarchy.

TABLE I: Redwood Nano vs. NVIDIA Jetson GPU performance comparison for Qwen3-0.6B LLM inference.

Comparison	Redwood Nano	NVIDIA GPU Jetson Orin Nano
Average Tokens/s (*)	12.1	28
Frequency (MHz)	250	1020
Memory Bandwidth (GBytes/sec)	16**	68
Memory Type	LPDDR4	LPDDR5

*Measured over 128 tokens generated.

**DRAM Memory BW for Redwood at peak achievable frequency of 250MHz, 4x 128-bit AXI4 streams.

A. Redwood顶线峰值解码吞吐量分析

我们首先计算峰值理论性能。操作级屋顶线源自 Qwen3-0.6B 解码图。该模型包含 28 个解码器层，隐藏层宽度为 1024，中间层宽度为 3072，16 个查询头，8 个 KV 头，头维度为 128，词汇表大小为 151,936。每一层中，INT8 线性形状分别为：Q : [2048, 1024]，K, V : [1024, 1024]，O : [1024, 2048]，gate, up : [3072, 1024]，down : [1024, 3072]。

矩阵引擎包含四个 tile，每个 tile 有 64 条 INT8 MAC 通道。将乘法和加法视为两次运算，其峰值速率为：

$$P_{\text{GEMV}} = 4 \times 64 \times 250 \times 10^6 \times 2 = 128 \text{ Gop/s.}$$

512 位 SIMD 数据通路每个周期可接受 32 个 BF16 或 16 个 INT32 操作数，峰值速率为

$$P_{\text{SIMD,BF16}} = 32 \times 250 \times 10^6 = 8 \text{ Gop/s},$$

$$P_{\text{SIMD,INT32}} = 16 \times 250 \times 10^6 = 4 \text{ Gop/s}.$$

对于每个未融合运算符 o_i ，我们计算算术运算量 Q_i 和总 DRAM 流量 P_i ，包括所有操作数读取和结果写入。其计算时间、内存时间和可达时间分别为

$$T_i^{\text{cmp}} = \frac{O_i}{P_i}, \quad T_i^{\text{mem}} = \frac{Q_i}{B}, \quad T_i = \max(T_i^{\text{cmp}}, T_i^{\text{mem}}).$$

运算符保持顺序执行，因为每个运算符都会消耗前一个运算符的结果。因此，完整 token 的界限为 $T^{\text{token}} = \sum T_i$ 。内存和计算资源可能在同一个运算符内重叠，但不会跨越运算符边界隐藏任何工作或流量。

TABLE II: Operation-level roofline for one Qwen3-0.6B decoder layer at context length 128. Each row may group adjacent unfused operators; its attainable time sums the individual operator maxima.

One-layer group	operator	Work (Mop)	DRAM (MB)	T_{cmp} (μs)	T_{mem} (μs)	$\sum T_i$ (μs)	Limit
	Input norm + QKV quant.	0.008	0.009	1.0	0.7	1.0	SIMD
	Q/K/V projections	8.397	4.247	68.6	302.5	303.2	DRAM
	Q/K norm, RoPE, and cache	0.034	0.042	4.2	3.0	4.7	SIMD
	QK and score scaling	0.530	0.158	6.1	11.2	12.3	DRAM
	Softmax and probability quant.	0.029	0.039	3.6	2.8	4.3	SIMD
	V quantization and PV	1.053	0.553	71.2	39.4	77.3	SIMD
	O projection and residual	4.206	2.124	34.7	151.3	152.0	DRAM
	MLP norm and input quant.	0.008	0.009	1.0	0.7	1.0	SIMD
	Gate and up projections	12.595	6.367	102.9	453.5	454.6	DRAM
	SiLU and gate multiply	0.022	0.031	2.7	2.2	3.6	SIMD
	Down projection and residual	6.307	3.177	51.6	226.2	227.3	DRAM
One decoder layer		33.189	16.755	347.5	1193.4	1241.4	Mixed

因此，单个解码器层的架构延迟为 1.241 毫秒（表 II）。28 个层共贡献 34.76 毫秒，而嵌入、最终归一化、语言模型头部和 argmax 操作又贡献了 11.26 毫秒。完整的词元移动 0.627 GB 的数据，需要 44.65 毫秒的 DRAM 服务总和以及 12.29 毫秒的算术服务总和。将每个运算符的根节点时间相加，得到：

$$T_{\text{token}} = 46.02 \text{ ms}, \quad R_{\text{roof}} = \frac{1}{T_{\text{token}}} = 21.73 \text{ tokens/s}.$$

内存服务时间是算术服务时间的 3.6 倍，这表明解码过程与内存使用量密切相关，正如预期。由于未考虑启动、同步、流水线填充/清空以及主机开销，测得的吞吐量仍低于架构上限。如果强制每个运算符内部的内存传输和执行进行串行化，则相应的保守上限为每个 token 56.94 毫

秒，即每秒 17.56 个 token。通过改进预取、软件调度以及控制路径中的硬件更新，仍可通过进一步优化软件和主机开销来降低吞吐量。

B. Redwood 性能、面积和功耗预测

我们估算了采用 1 GHz 逻辑时钟的配置的架构顶线，我们认为考虑到 FPGA 时序，该时钟频率是合理的。顶线模型将吞吐量限制在权重传输路径（DRAM → NoC → 边缘 DMA → 网状交换机 → 计算）中最慢的阶段，因此我们计算每个阶段的可持续带宽并取最小值。关键在于，DRAM 数据速率（每引脚 3900 Mb/s）由内存设备决定，不会随架构时钟频率而扩展；1 GHz 时钟频率加速了片上架构、SIMD 引擎和 MAC 阵列。假设带宽与 Nvidia Jetson Orin Nano 相同，则 tile 入口路径仍然是最窄的阶段，因此该设计仍然受限于内存传输，吞吐量约为 64 GB/s。在此场景下，架构上限约为 95 个 token/秒，这是基于 128 个生成的 token 的平均值。相比目前约 21 个 token/秒的上限，性能提升主要来自启用三个控制器并提高逻辑时钟频率，这使得总的 tile 入口带宽从 16 GB/秒提升至 64 GB/秒，并将计算引擎的数量增加了四倍。在非理想配置下，模型的执行并非总能与数据传输重叠。我们对 Qwen 在 FPGA 上的执行进行了分析，并根据更高的时钟频率、更大的内存带宽以及更少的硬件限制带来的更优任务调度，仔细调整了每个步骤。我们最保守的预测表明，在不更改软件栈的情况下，当前的 ASIC 设计在 128 个生成的 token 上平均可实现约 49 个 token/秒的性能。

我们根据三星 8 纳米工艺（与 NVIDIA Jetson Orin Nano 所用的工艺相当）来预测 Redwood 的面积和功耗。物理面积的估算采用了一种标准的自下而上的门等效 (GE) 方法，该方法针对三星 8 纳米物理设计规则进行了调整。通过将 200 万个组合逻辑单元和 50 万个时序寄存器按其平均相对门尺寸加权，将标准单元数量转换为 GE 单位。它们的和即为原始标准单元面积，代表不包含互连间隙的有源硅面积。我们为测试逻辑设计增加了 15% 的开销。70% 的布局利用率则为金属布线、电源轨和去耦电容预留了空间。最后，我们为时钟树综合中继器、时序收敛单元和周边结构增加了 20% 的面积开销。因此，Redwood Nano 预计在三星 8 纳米工艺下的 NPU 模块面积约为 2.88 平方毫米。

我们通过分离动态功耗和静态功耗来估算 Redwood 的总功耗。核心动态功耗遵循公式 $P^{dyn} = \alpha C V^2 f$ ，而核心静态功耗用 P^{stat} 表示。在三星 8nm 工艺下，目标频率为 1.0 GHz，标称核心电压为 $V^{core} = 0.75V$ ，每个 tile 包含 200 万个逻辑单元、50 万个触发器和 512KB SRAM，在 Qwen3-0.6B 的平均开关活动下，其动态功耗约为 0.958 W。静态漏电约为 $P^{stat} \approx 0.07 W$ 。加上剩余的 SoC 和时钟管理组件，芯片端总功耗约为 $P^{total} \approx 1.335 W$ 。该数值与基于 FPGA 的测试结果一致，并且是上限值，因为我们尚未考虑 Redwood 架构原生支持的积极时钟和电源门控所带来的节能效果。

在 Jetson 上运行相同的应用程序，我们实现了每秒 28 个 token 的处理速度。Jetson 的 CPU 和 GPU 核心在启用融合功能后的平均功耗为 2.59 瓦。为保证公平性，上述功耗估算均包含主机和加速器计算，但不包括内存控制器和其他外围组件。因此，Redwood 在采用三星 8 纳米工艺的同类产品上的性能和功耗预测显示，性能提升 1.75 倍，功耗降低 1.9 倍，每瓦性能提升 3.4 倍，同时显著缩短了设计时间和上市时间。表 III 总结了 Redwood Nano 与 NVIDIA Jetson Orin Nano 相比的预计性能、面积和功耗。

TABLE III: Redwood Nano vs. NVIDIA Jetson Orin Nano power, area, and performance for Qwen3-0.6B LLM inference on a Samsung 8 nm-class process.

Comparison	Redwood Nano	NVIDIA GPU Jetson Orin Nano
Average Tokens/s *	49 (1.75×)	28
Power (W)	1.335 (1.9×)	2.59
Area (mm ²)	2.88	NA
Tokens/s per Watt	36.7 (3.4×)	10.8

*Projected over 128 tokens generated (ASIC).

五、Architect Labs 系统

在不到两周的时间内，整个 Redwood 系统完成了设计、验证，并达到了综合和物理设计就绪状态，最终部署为 Redwood Nano FPGA 配置。该加速器完全从零开始研发，没有使用任何预先存在的加速器 IP。这不仅得益于我们自主训练模型、构建代理框架和开发 AI 原生 EDA 工具，更得益于我们对从软件到芯片的硬件设计流程进行了根本性的重新思考。传统的芯片设计生命周期高度顺序化，从架构定义到最终流片，按阶段推进。在此过程中，相关组件会被“冻结”，变更要么是临时性的，要么是为下一代产品预留的。硬件团队采用流水线式开发模式，使得在版本 N 冻结后，相应的团队立即开始版本 N+1 的开发（图 9）。这使得大型硬件公司能够以 9-12 个月的周期发布新硬件。

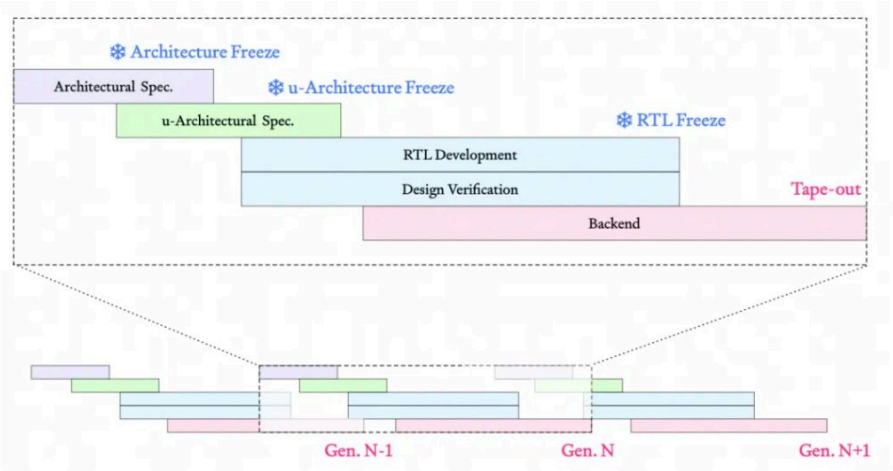


Fig. 9: Representative life cycle of a traditional ASIC program.

虽然其吞吐量可能尚可接受，但这种顺序方法的延迟使得真正的软硬件 (HW-SW) 协同设计变得不切实际。在当前的 AI 环境中，当架构定义完成、RTL 设计和验证工作启动时，新的模型可能会使数月的优化成果付诸东流。因此，硬件团队必须预测工作负载的发展方向，并添加通用功能作为应对措施。

Architect Labs 的流程尽可能地自动化和并行化，无需“冻结”设计，从而实现灵活的架构探索和端到端实现。它基于 Architect Labs 平台 (ALP) 构建，ALP 是我们内部用于端到端芯片设计的平台（图 10）。一旦设计意图被捕获到 ALP 中，自动化流程就会生成 RTL、UVM 文档、SVA 断言、形式化证明和其他工件。在规范以下阶段无需人工干预；专家会在整个项目生命周期中维护 ALP，并根据功能、面积、性能、时序和功耗方面的反馈来调整规范或设计意图。



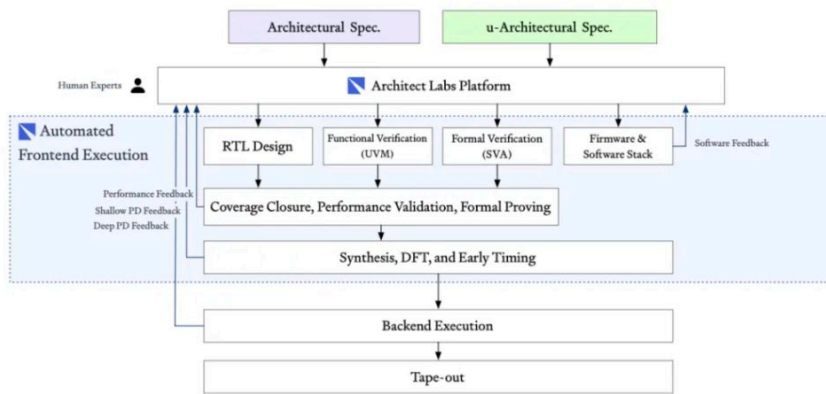


Fig. 10: Architect Labs Platform and end-to-end chip design flow.

这种自动化流程的一个关键优势在于，人和智能体可以并行探索多种架构方案。对于 Redwood 项目，每个架构迭代都在 48 小时内完成重新生成、重新验证并重新部署到 FPGA。从初始规范到最终的 RTL 代码、验证、固件、自定义内核以及时序收敛，整个设计周期耗时两周，所有模块的代码和功能覆盖率均达到 95%。第三周将目标工作负载（即新的 AI 模型）部署到 FPGA 上；完整的为期三周的项目时间表如图 11 所示。

图 12 显示了 AI 系统在项目生命周期内的代码库合并提交历史记录。在目标工作负载上线并持续优化 Redwood 的固件和内核期间，AI 系统在一天内达到了 115 次合并提交的峰值。

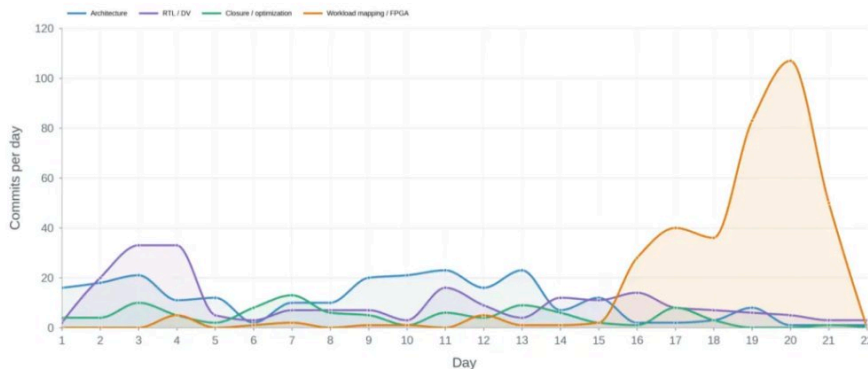


Fig. 12: Redwood repository commit activity.

A. 自动化设计验证和覆盖率收敛

Redwood 的测试平台生成、测试用例开发和覆盖率收敛均已实现完全自动化。传统的“用于验证的 AI 代理”通常需要工程师通过聊天来自动化测试用例开发和测试平台构建。这种方法仍然需要大量的人工投入，并且无法随着芯片复杂性的增加而扩展，因此并未显著缩短端到端 ASIC 开发周期。相比之下，Redwood 的所有测试平台、测试用例、形式化工件和仿真均由 ALP 使用 AI 和编译器方法自动生成，无需人工参与设计验证。

遵循行业标准验证实践，我们采用 UVM 技术以及现代形式化方法。我们开发了自主研发的形式化引擎的首个版本，该引擎能够根据人工编写的规范生成每个验证环境的各个部分。验证流程测量并自动优化了覆盖率和性能指标。每个模块都实现了 95% 的代码和功能覆盖率。在将第一个从仿真环境发送到 FPGA 的 RTL 代码时，未发现任何错误。迄今为止，我们的验证方法尚未遇到任何在验证环境中遗漏但在实际硬件中发现的错误。随着我们技术的扩展，我们预计验证的严谨性将随着可用计算资源的增加而扩展，而不仅仅是取决于团队规模和 EDA 工具许可证的数量。我们期待在未来的公告中分享更多信息。

B. 设计与优化

由于我们的流程中 RTL 设计完全自动化，系统能够在相同时间内探索比人类团队大一个数量级的微架构搜索空间。以我们的 SIMD 引擎为例，它在多个向量通道上执行降精度浮点和整数运

算。虽然单个通道的开发和复制都很简单，但诸如 reduce、max 和 min 之类的归约操作会跨越所有通道。通过在更长的时间范围内运行更多系统实例，我们可以发现大量新的候选方案。在图 13 和图 14 所示的示例中，我们的 AI 系统在多天内遍历了性能-面积-时间搜索空间，在保持代码覆盖率和验证严谨性的同时，进行设计、验证和优化。



Fig. 13: Autonomous SIMD engine exploration path.

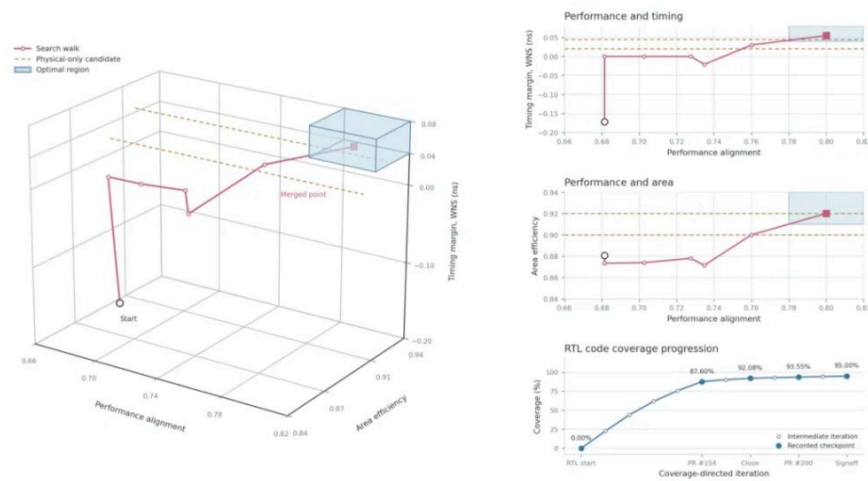


Fig. 14: SIMD engine micro-architectural exploration and optimization for performance, area, and timing, and code coverage.

以往的自动化微架构探索方法通常仅限于位宽调整或寄存器重排等更改。而在这里，生成的RTL候选方案可以采用截然不同的控制路径、数据路径和状态机，并能够自由地寻找超越人类认知最优解的解决方案。随着技术的扩展，我们预期探索质量将受限于可用计算能力而非人类洞察力；随着计算能力的提升，系统将能够发现超出最优秀人类设计师认知极限的架构。

C. 固件生成和内核优化

ALP 的一项优势在于，它支持在编写任何 RTL 或验证文档之前，对所有系统软件（包括固件、内核和性能模型）进行协同开发。这使得架构师能够在最终确定实现方案之前，做出明智的设计决策。我们的 AI 系统编写并测试了启动 SoC 和运行 Qwen 推理所需的所有固件和内核。根据运行时环境的不同，系统软件会根据 ALP 预测结果、周期精确的 RTL 仿真或现有的 FPGA 构建进行测试。我们设计了一个内部定制的仿真环境，该环境将 FPGA 访问复用到数百个并发代理，使它们能够运行实验、共享性能结果，并在数天内无需人工干预即可迭代。在某些情况下，AI 系统发现了我们的专家尚未考虑或深入探索的优化和架构改进。该系统还帮助我们发现了下一代 Redwood 的新硬件特性，从而可以对新设计进行多次并行迭代。

从RTL和性能模型仿真迁移到我们内部的FPGA环境，将优化运行时间从15小时缩短到大约15-30分钟。随着人工智能在设计流程中实现更多自动化，我们相信基于FPGA的仿真将对加速性能验证和ASIC路线图的制定至关重要。

D. 递归式自改进



最终，我们在 Redwood 上部署了 Qwen3，并将其作为我们 AI 系统中的推理端点。通过重复采样，AI 模型在其多个操作中发现了多项时间优化和内核优化，且推理成本为零。我们认为这是递归式自改进的最早演示之一：一个 AI 系统设计了一个 AI 加速器，在其上部署了一个 AI 模型，并利用该模型改进了下一代加速器。如今，能够自主设计硬件的 AI 系统的能力与能够实际部署在此类硬件上的 AI 模型的能力之间存在差距，如图 15 所示。随着我们的 AI 系统不断扩展到更复杂的硬件设计，并且我们缩小了这种差距，持续的 AI 驱动的现代硬件改进将成为 AI 进步的最大驱动力之一。我们认为这是衡量递归式自改进最清晰的指标。

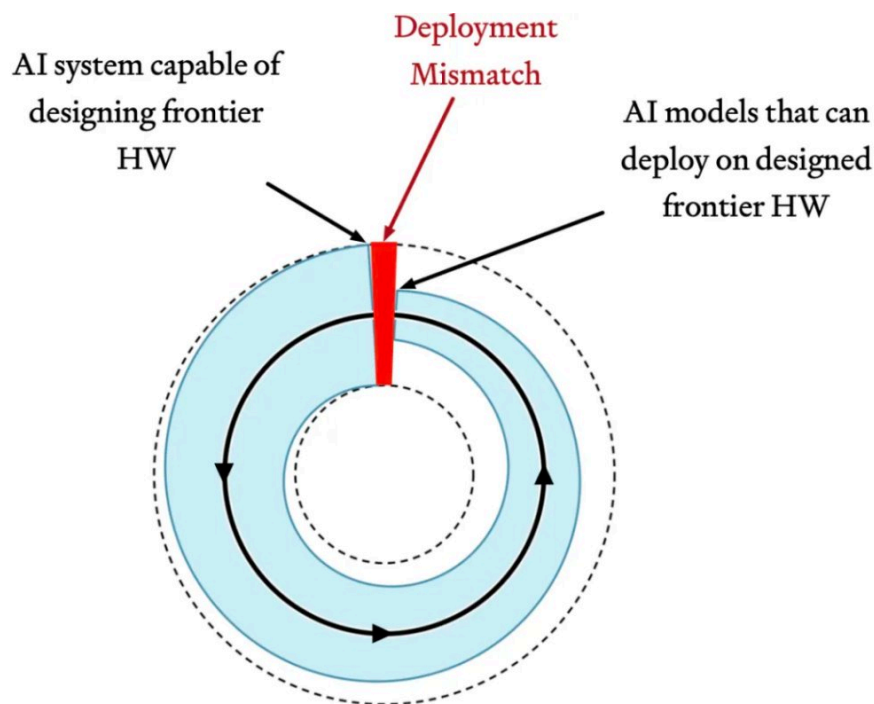


Fig. 15: Requirements for recursive self-improvement.

六、结论

我们展示了 Redwood，一款由人工智能系统从零开始设计、验证和部署的可编程加速器，整个过程耗时不到两周。该系统实现了 95% 的代码和功能覆盖率，Redwood Nano 在 AMD Versal FPGA 上运行 Qwen3-0.6B 算法，峰值和平均速度分别为 13 个 token/秒和 12.1 个 token/秒。根据这些 FPGA 测量结果进行校准后，我们基于三星 8 纳米工艺的预测结果为：功耗 1.335 瓦时，速度可达 49 个 token/秒，相比我们测量的 Jetson Orin Nano 基准，每瓦性能提升了 3.4 倍。

在传统的顺序芯片设计流程中，人工智能辅助并未显著缩短端到端开发时间。我们的人工智能系统采用了一种正交方法：以高级规范为真理之源，架构、RTL、验证、固件和内核均基于此进行协同设计和优化。这使得架构变更能够在 48 小时内重新验证并部署到硬件上。

未来的工作将着重于弥合内存容量与性能上限之间的差距，将 Redwood 扩展到更大的模型和架构，并通过物理设计、流片和芯片后验证来扩展 AI 系统。我们的最终目标是通过递归式自我改进实现智能的泛滥：随着 AI 模型和可用计算能力的提升，系统可以探索和优化更多 AI 硬件设计，从而产生更高效的计算，进一步加速下一代 AI 和硬件的开发。

本文来源：[半导体行业观察](#)

风险提示及免责声明

市场有风险，投资需谨慎。本文不构成个人投资建议，也未考虑到个别用户特殊的投资目标、财务状况或需要。用户应考虑本文中的任何意见、观点或结论是否符合其特定状况。据此投资，责任自负。

写评论

请发表您的评论



表情



图片

发布评论

